



Lab Assignment 4

Course: Higher Level Programming Language Lab

Course Code: CSEL-3204

Submitted To

Dr. MD. Manowarul Islam

Professor

Department of CSE

Jagannath University

Submitted By

Maisha Binte Monir

ID: B210305016

Session: 2021-22 (CSE-13)

Task 1: Create arrays including 1D arrays, zero matrices, identity matrices, and evenly spaced arrays.

a) Create a 1D NumPy array containing numbers from 1 to 20.

b) Create a 5×5 matrix filled with zeros.

c) Create a 4×4 identity matrix.

d) Create an array with evenly spaced values between 10 and 50.

```
import numpy as np
a = np.arange(1, 21)
print(a)
b = np.zeros((5, 5))
print('Zero matrix:\n', b)
c = np.eye(4)
print('Identity matrix:\n', c)
d = np.linspace(10, 50, 9)
print('Evenly spaced matrix: ', d)
```

Output:

```
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20]
```

Zero matrix:

```
[[0. 0. 0. 0. 0.]
```

```
[0. 0. 0. 0. 0.]
```

```
[0. 0. 0. 0. 0.]
```

```
[0. 0. 0. 0. 0.]
```

```
[0. 0. 0. 0. 0.]]
```

Identity matrix:

```
[[1. 0. 0. 0.]
```

```
[0. 1. 0. 0.]
```

```
[0. 0. 1. 0.]
```

```
[0. 0. 0. 1.]]
```

Evenly spaced matrix: [10. 15. 20. 25. 30. 35. 40. 45. 50.]

Task 2: Display array attributes such as shape, dimensions, data type, and size.

```
array2 = np.array([[20, 30, 40, 50, 60],
[26, 37, 27, 16, 28],
[29, 94, 27, 88, 24]])
print('The array: ')
print(array2)
print('Array2 shape:', np.shape(array2))
print('Array dimension: ', np.ndim(array2))
print('Array data type: ', array2.dtype)
print('Array number of elements: ', np.size(array2))
```

Output:

The array:

[[20 30 40 50 60]

[26 37 27 16 28]

[29 94 27 88 24]]

Array2 shape: (3, 5)

Array dimension: 2

Array data type: int64

Array number of elements: 15

3. Extract rows, columns, and subarrays from a given NumPy array. a) Extract the first row. b) Extract the last column. c) Extract the subarray containing:

#a

```
first_row = array2[0]
print('First row: ', first_row)
```

#b

```
last_column = array2[:, -1]
print('Last column: ', last_column)
```

#c

```
sub = array2[0:2,1:3]
print('A subarray:\n', sub)
```

Output:

First row: [20 30 40 50 60]

Last column: [60 28 24]

A subarray:

[[30 40]

[37 27]]

4.Modify array values using indexing.

```
array2[[2]]=30
print('Modified array2 (in 3rd row): ')
print(array2)
```

Output:

Modified array2 (in 3rd row):

[[20 30 40 50 60]

[26 37 27 16 28]

[30 30 30 30 30]]

Perform Addition, Subtraction, Multiplication, Division, Dot Product

```
first = np.array([10, 20, 30])
second = np.array([2, 4, 5])

print('Array 1: ',first)
print('Array 2: ',second)

print("Addition:", np.add(first, second))
print("Subtraction:", np.subtract(first, second))
print("Multiplication:", np.multiply(first, second))
print("Division:", np.divide(first, second))
```

```
print("Dot Product:", np.dot(first, second))
```

Output:

Array 1: [10 20 30]

Array 2: [2 4 5]

Addition: [12 24 35]

Subtraction: [8 16 25]

Multiplication: [20 80 150]

Division: [5. 5. 6.]

Dot Product: 250

5. Calculate mean, median, standard deviation, maximum, and minimum values.

```
cal = np.array([[10,20,30],  
[40,50,60]])
```

```
print(cal)
```

```
print('Mean value: ',np.mean(cal))
```

```
print('Median value: ',np.median(cal))
```

```
print('Standard Deviation value: ',np.std(cal))
```

```
print('Maximum value: ',np.max(cal))
```

```
print('Minimum value: ',np.min(cal))
```

Output:

```
[[10 20 30]
```

```
[40 50 60]]
```

Mean value: 35.0

Median value: 35.0

Standard Deviation value: 17.07825127659933

Maximum value: 60

Minimum value: 10

6. Perform matrix operations including addition, multiplication, transpose, and determinant.

```
import numpy as np

A = np.array([[22,48],
              [35,28]])

B = np.array([[24,27],
              [26,43]])

print("Matrix A:\n", A)
print("Matrix B:\n", B)

print("Addition:\n", A + B)
print("Multiplication (element-wise):\n", A * B)
print("Matrix Multiplication:\n", np.dot(A, B))
print("Transpose of A:\n", A.T)
print("Determinant of A:", np.linalg.det(A))
```

Output:

Matrix A:

[[22 48]

[35 28]]

Matrix B:

[[24 27]

[26 43]]

Addition:

[[46 75]

[61 71]]

Multiplication (element-wise):

[[528 1296]

[910 1204]]

Matrix Multiplication:

[[1776 2658]

[1568 2149]]

Transpose of A:

[[22 35]

[48 28]]

Determinant of A: -1064.0000000000002

7. Reshape arrays into different dimensions and explain reshaping.

```
arr = np.array([10,20,30,40,50,60])
print("Original array:", arr)
arr1 = arr.reshape(2, 3)
print('Reshaped into 2x3 matrix:\n', arr1)
arr2 = arr.reshape(3, 2)
print('Reshaped into 3x2 matrix:\n', arr2)
arr3 = arr.reshape(2, -1)
print('Reshaped automatically using -1:\n', arr3)
```

Output:

Original array: [10 20 30 40 50 60]

Reshaped into 2x3 matrix:

[[10 20 30]

[40 50 60]]

Reshaped into 3x2 matrix:

[[10 20]

[30 40]

[50 60]]

Reshaped automatically using -1:

[[10 20 30]

[40 50 60]]

8. Analyze student marks using NumPy arrays.

```
students = ["Student1", "Student2", "Student3", "Student4",
"Student5"]

subjects = ["Math", "Physics", "CSE"]

marks = np.array([
    [78, 85, 90],
    [65, 70, 72],
    [88, 92, 95],
    [55, 60, 58],
    [80, 75, 82]
])

print("                ", end="")

for sub in subjects:
    print(f"{sub:10}", end="")
    print()

for i in range(len(students)):
    print(f"{students[i]:10}", end="")
    for j in range(len(subjects)):
        print(f"{marks[i][j]:10}", end="")
    print()

print("\nSubject-wise Mean:", np.mean(marks, axis=0))
print("Subject-wise Max:", np.max(marks, axis=0))
print("Subject-wise Min:", np.min(marks, axis=0))

print("\nStudent-wise Mean:", np.mean(marks, axis=1))
print("Student-wise Total:", np.sum(marks, axis=1))
print("Student-wise Max:", np.max(marks, axis=1))
print("Student-wise Min:", np.min(marks, axis=1))
```



```

print("\nOverall Mean:", np.mean(marks))
print("Overall Median:", np.median(marks))
print("Standard Deviation:", np.std(marks))

totals = np.sum(marks, axis=1)
top_student = np.argmax(totals)

print("\nTotal Marks:", totals)
print("Top Student Index:", students[top_student])
print("Highest Score:", totals[top_student])

```

Output:

Math	Physics	CSE	
Student1	78	85	90
Student2	65	70	72
Student3	88	92	95
Student4	55	60	58
Student5	80	75	82

Subject-wise Mean: [73.2 76.4 79.4]

Subject-wise Max: [88 92 95]

Subject-wise Min: [55 60 58]

Student-wise Mean: [84.33333333 69. 91.66666667 57.66666667 79.]

Student-wise Total: [253 207 275 173 237]

Student-wise Max: [90 72 95 60 82]

Student-wise Min: [78 65 88 55 75]

Overall Mean: 76.33333333333333

Overall Median: 78.0

Standard Deviation: 12.348639151294725

Total Marks: [253 207 275 173 237]

Top Student Index: Student3

Highest Score: 275

9. Calculate averages, identify highest marks, and filter students based on conditions. (Using the previous dataset)

```
student_avg = np.mean(marks, axis=1)
print("Student-wise Average:", student_avg)

subject_avg = np.mean(marks, axis=0)
print("Subject-wise Average:", subject_avg)

student_total = np.sum(marks, axis=1)
print('Student wise total: ', student_total)

print("\nHighest Mark:", np.max(marks))
print("Student-wise Highest Marks:", np.max(marks, axis=1))
print("Subject-wise Highest Marks:", np.max(marks, axis=0))

excellent = np.array(students)[student_avg >= 80]
print("\nStudents who got A+ on avg:", excellent)

failed = np.array(students)[np.any(marks < 60, axis=1)]
print("Students who got poor result:", failed)

#students ranking
rank_indices = np.argsort(student_total)[::-1]
print("\nStudent Ranking:")
for i, idx in enumerate(rank_indices):
```

```

rank = i + 1
if rank == 1:
    suffix = "st"
elif rank == 2:
    suffix = "nd"
elif rank == 3:
    suffix = "rd"
else:
    suffix = "th"

print(f"{rank}{suffix} Place: {students[idx]} (Total:
{student_total[idx]})")

print()

# student grades
grades = []
for a in student_avg:
    if a >= 80:
        grades.append("A+")
    elif a >= 70:
        grades.append("A")
    elif a >= 60:
        grades.append("B")
    elif a >= 50:
        grades.append("C")
    else:
        grades.append("F")
grades = np.array(grades)
for i in range(len(students)):
    print(students[i], "->", "Avg:", round(student_avg[i], 2), "Grade:",
grades[i])

```

Output:

Student-wise Average: [84.33333333 69. 91.66666667 57.66666667 79.]

Subject-wise Average: [73.2 76.4 79.4]

Student wise total: [253 207 275 173 237]

Highest Mark: 95

Student-wise Highest Marks: [90 72 95 60 82]

Subject-wise Highest Marks: [88 92 95]

Students who got A+ on avg: ['Student1' 'Student3']

Students who got poor result: ['Student4']

Student Ranking:

1st Place: Student3 (Total: 275)

2nd Place: Student1 (Total: 253)

3rd Place: Student5 (Total: 237)

4th Place: Student2 (Total: 207)

5th Place: Student4 (Total: 173)

Student1 -> Avg: 84.33 Grade: A+

Student2 -> Avg: 69.0 Grade: B

Student3 -> Avg: 91.67 Grade: A+

Student4 -> Avg: 57.67 Grade: C

Student5 -> Avg: 79.0 Grade: A